USC Viterbi
School of Engineering

Reinforcement Learning Card Game Agent

EE 547 – FINAL PROJECT REFERENCE DECK

Wanshi Cao,
Ping-Hsi Hsu,
Zihan Shen

Fall 2025

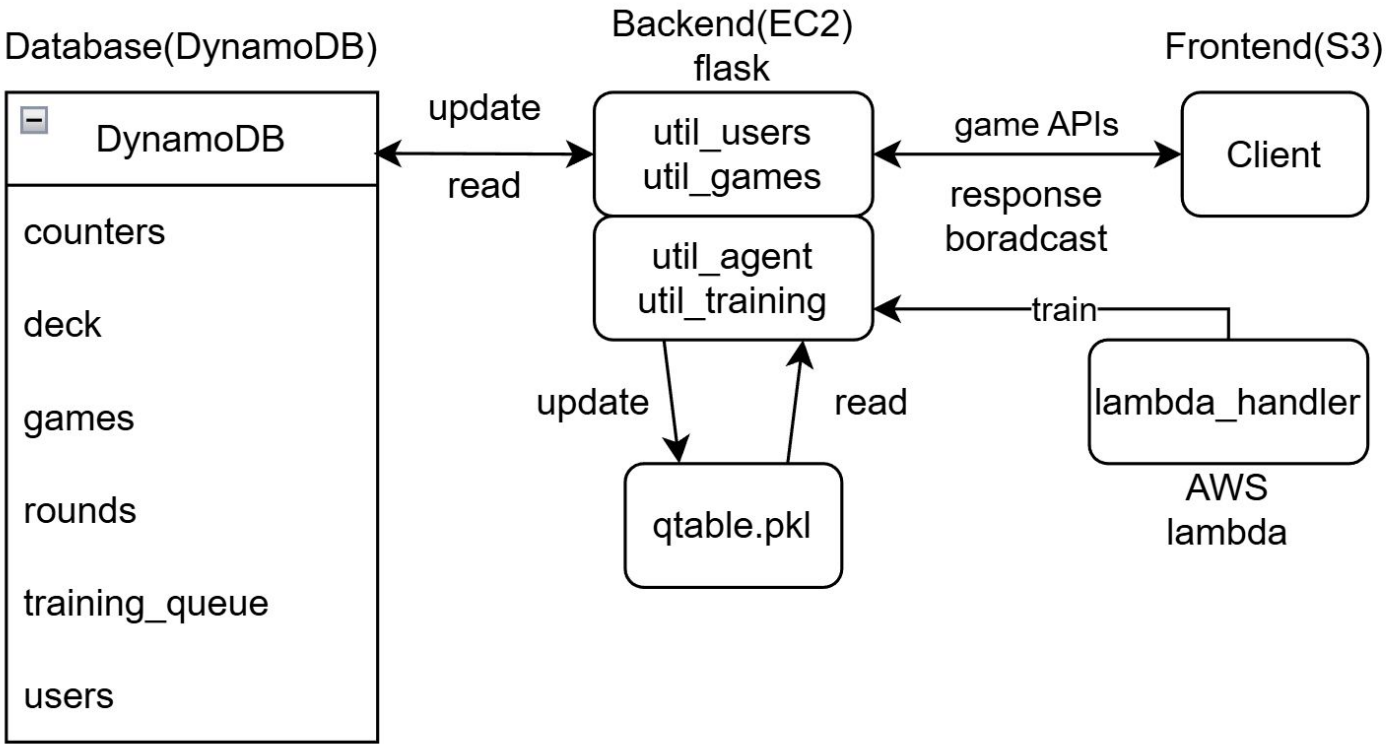
PROJECT SUMMARY

- Traditional card game is usually lack of intelligent opponent, systematic learning and scalability. Deploying an AI-powered system is also relatively expensive.
- This project construct an AWS based application which support a three round card game (both PvP and PvE), and uses game data to support Q-table update of a reinforcement learning agent on the specific card game.

ARCHITECTURE SUMMARY

- DynamoDB for data storage, EC2 for server running, and S3 for frontend architecture. Lambda is used to update Q-table.
- Frontend handle user interface and send action information to backend server, backend server handle information and provide feedback and broadcast.
- Both frontend, backend and reinforcement learning agent is property implemented.

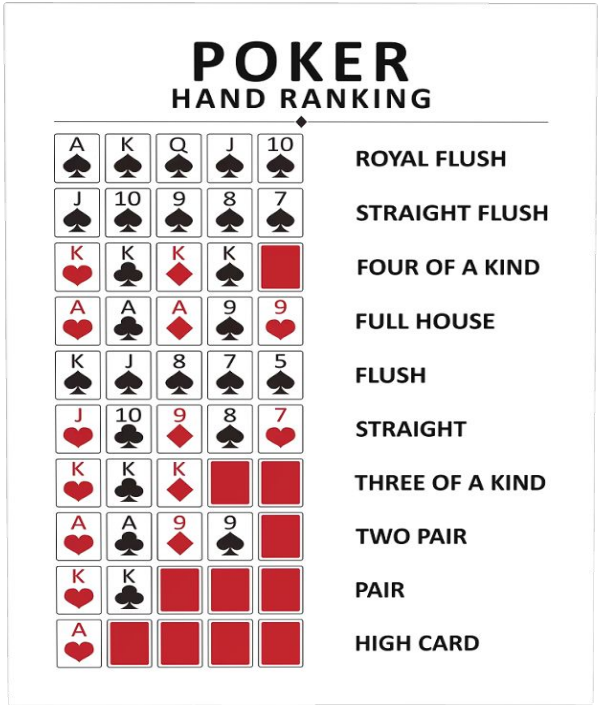
ARCHITECTURE SUMMARY



BACKGROUND - Game Rule

- Simplified 5-Card Draw Poker with BO3 (first to 2 wins)
  - Deal: Each player receive 5 cards.
  - Redraw: Player can choose any cards to redraw.
  - Showdown: Hands are compared with standard Poker rankings.
- Limited resources: Each player has a Global Budget of 7 Redraws for the entire match.

BACKGROUND - Game Rule



DEPLOYMENT

- Source Code saved on Github, and entire system is deployed on AWS server.
- We use S3 for front end hosting, EC2 to run the Flask server and Q-learning agent, and DynamoDB for data storage.
- The backend runs with Python environment without containers, and deployed with AWS console and Python script.
- Detailed setup instructions included in readme file on github.
- Deployment process was fully completed.

TIMELINE – RETROSPECTIVE

Task	Expected Date	Actual Date	Detail
Revised Proposal	Nov 9, 2025	Nov 9, 2025	N/A
Backend API/Server	Nov 30, 2025	Nov 30, 2025	Does end up needing some further debug
Backend continue	Dec 3, 2025	Dec 4, 2025	Broadcast feature
Frontend	Dec 9, 2025	Dec 9, 2025	N/A
QLearner	Nov 30, 2025	Nov 30, 2025	Local Training
Deployment	Dec 9, 2025	Dec 9, 2025	Add to EC2 Server, Lambda Training

REFERENCES - Library documentation

- Flask API library:
  - https://flask.palletsprojects.com/en/stable/
  - https://flask-socketio.readthedocs.io/en/latest/
- Bcrypt Encode Library
  - https://pypi.org/project/bcrypt/

FRONTEND

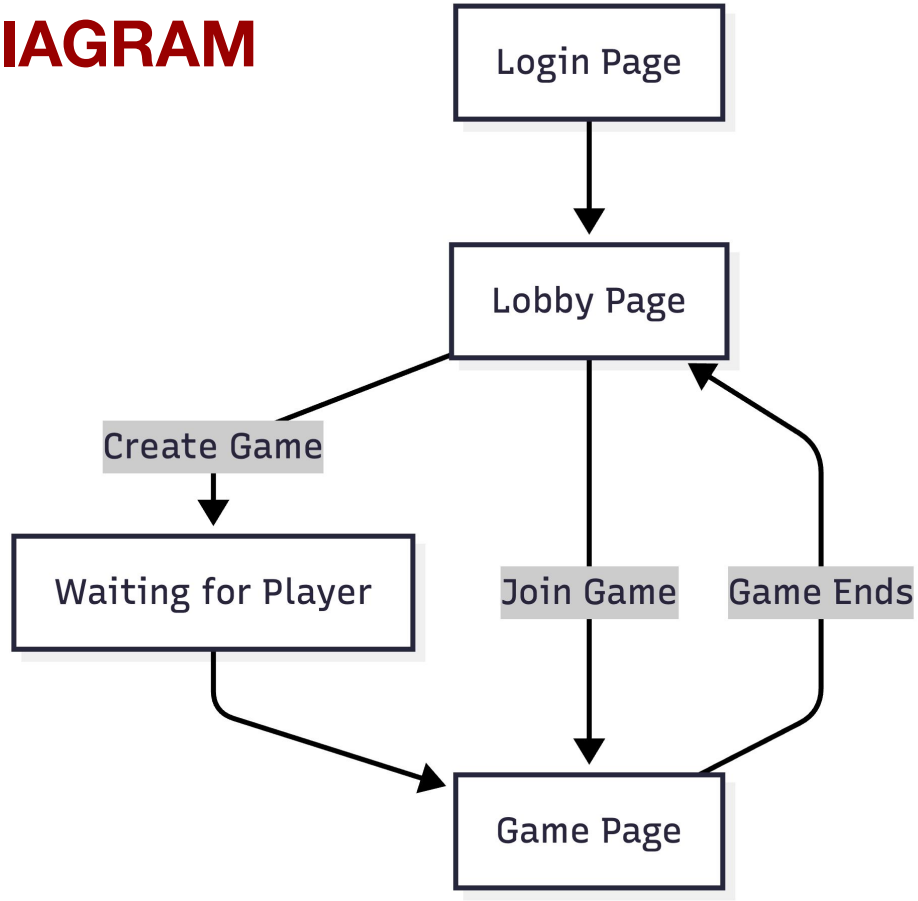
FRONTEND OVERVIEW

- Built with vanilla HTML/CSS/JavaScript + Flask.socketio frontend client
- Hosted as static files from Flask /static
- No bundler; assets served directly
- Integrates with backend via:
  - REST API (login, create game, join game, redraw, confirm...)
  - Flask.socketio (real-time updates: listen, system, round events)
  - JWT passed from frontend → backend for authentication

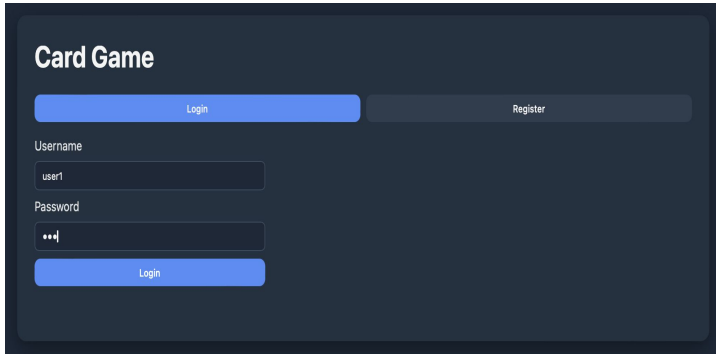
USER ROLES

- Role: Player
  - Who this is:
    - A user who can choose to play either against another player or against an AI bot.
  - What they can do:
    - Choose PVP (vs player) or PVE (vs AI bot)
    - Create a game or join a game
    - View their hand and redraw cards
    - Play and confirm moves
    - Receive real-time updates via Socket.IO
  - Relevant pages / features:
    - Login Page
    - Lobby Page
    - Game Page (hand display, redraw, confirm, real-time events)

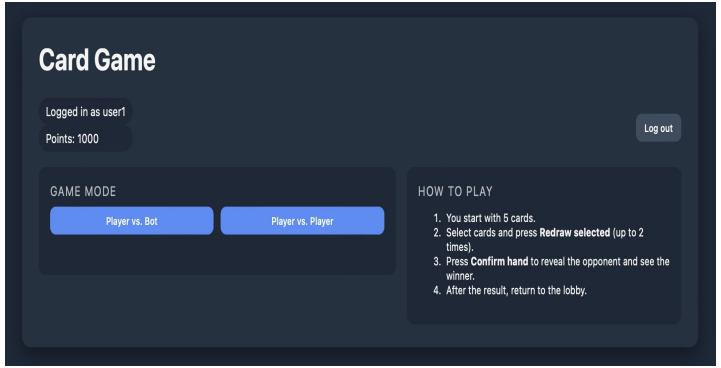
USER FLOW DIAGRAM



PAGE:

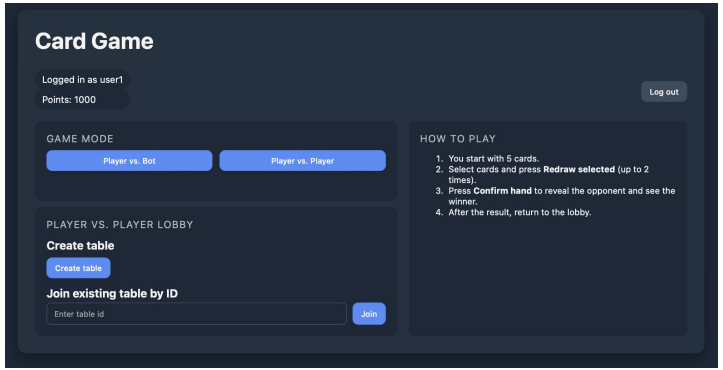


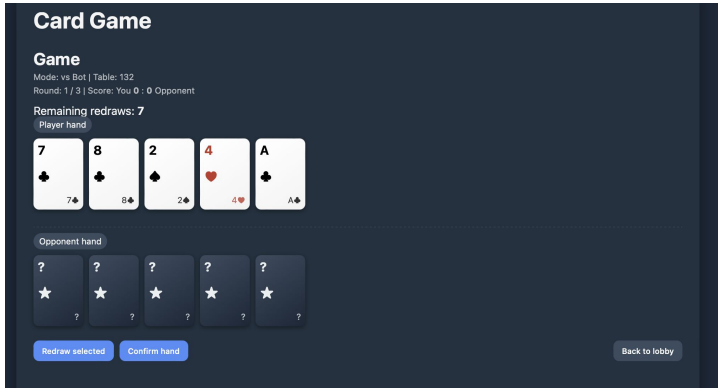
- [login page summary]
- Users register account
- Users enter username and password to authenticate.
- A JWT token is generated and stored locally.
- Redirects automatically to Lobby upon successful login.



- [lobby page summary]
- Player selects mode: PVP or PVE.
- Central hub where users choose to Create Game or Join Game.
- Displays game ID after creation.
- Allows entering an existing game ID to join PVP.
- PVE mode starts instantly (no waiting for other players).

PAGE:





- [Game page summary]
- Both players view their hand cards.
- Players can select, redraw cards, and confirm moves.
- Real-time updates.
- Displays round number, remaining redraws, and game status.

BACKEND

BACKEND SUMMARY

- The backend uses Python 3.13.5, flask framework with flask_socketio to broadcast message.
- Flask server is deployed on EC2 instance directly, with 4 different util files.
- Flask handle all API requests and uses direct response/private room broadcast to provide information/update game status
- Database used is DynamoDB (detail next slide)
- Service triggered when client send information

  (request, user action etc.)

BACKEND SUMMARY - Data Storage

users

username	pwHash	points	status

counter

name	ct

games

gameid	mode	p1	p1_score	p2	p2_score	status

rounds

rid	gameid	p1_confirm	p1_deckid	p1_hand	p1_rd
p2_confirm	p2_deckid	p2_hand	p2_rd	round_num	status

deck

deckid	decision	gameid	hand	player	rd_remain	rid	score_diff

training_queue

gameid	status	timestamp

OUTSIDE DATA SOURCE / API

- No outside API used.
- The Reinforcement learning agent’s Q-table(pkl) is based on manual upload from Wanshi Cao’s training on local machine.

PERFORMANCE AND SCALABILITY

- We did not apply parallelism testing, but we could expect this being a problem due to Flask's default single thread operation.
- Usage testing is applied under the condition of both PvP and PvE. The system works well under normal load access.
- A notable latency and delay occurs when multiple client request the service.

DEVOPS / SYSTEM OPERATIONS

- CI/CD
  - This project does not implement a full CI/CD pipeline
  - We uses manual update backend to EC2 instance and frontend to S3 bucket.
- Monitoring:
  - The train feature include writing a json log output about training statistic (time, game ids, update count etc.)

Error Handling

- 400 Bad Request
  - Missing username/password in registration/login
  - Username exist in registration
  - Invalid password
  - User in already game when join/create game
  - Missing ‘gameid’ when joining room

Error Handling

- 401 Unauthorized
  - JWT token authorization failed
- 404 Not Found
  - Invalid user
  - invalid gameid
  - roundid not found
- 500 Server Error
  - Model training error

MACHINE LEARNING

ML OVERVIEW

- Model Type: Reinforcement Learning (RL) agent using Q-Learning with State-Action Value Function.
- Purpose: Acts as an intelligent opponent in PvE mode.
- Framework: Custom-trained mode built from scratch in Python.

ML INTEGRATION

- Where inference occurs: Flask on AWS EC2 and triggered during the game_confirm phase in PVE mode.
- Execution Flow:
  - Input: Current hand + Game State (Score, Remaining redraws)
  - Process: Convert raw cards to State Tuple
  - Output: List of card indices to redraw
- Synchronous since the inference is extremely lightweight.

ML DATA AND TRAINING

- Data source:
  - Self-Play Simulation (Local)
  - Real User Data (AWS DynamoDB)
- Training Environment:
  - Automation: AWS Lambda triggers a daily training job.

Fetch pending games -> Update Q-table -> Reload Agent

Deployment: Loaded into memory on server startup for fast access

ML PERFORMANCE

- Latency: Ultra-low inference latency (microseconds level) due to dictionary lookup.
- State space optimization: Reduced complexity by abstracting specific cards into Ranks & Masks (Suited/Straight).

  C(52,5) about 2.6 million -> 100,000

API

API SUMMARY

- Type: REST API & Flask_socketio
- Framework: Flask (Python) Format: Json
- Primary Endpoints:
  - Auth: POST /user/register
  - User: GET /user/points
  - Training: POST /model/train
  - Game: GET /game/creategame
  - Real-time: flask.socketio used for broadcasting game state updates

API SECURITY AND ACCESS

- A '***HS256***' signature JWT is mandatory for any of the game utility API calls.
- When the user register, the backend saved a salted and hashed version of the password by '***bcrypt***' package. This is also used to compare password when user login.
- API calls check this JWT in the header. SocketIO request keep this token in the content since it has no header.

API ENDPOINTS - user

- POST /user/register [username, password]
  - No Authorization
  - Action: write encoded information to user table
  - Return message ‘Register success’
- POST /user/login [username, password]
  - No Authorization
  - Action: check user identity and return JWT if valid
  - Return token <JWT token>
- GET /user/points
  - Authorization: header Bearer carries JWT
  - Action: validate user identity, return user’s points
  - Return points

API ENDPOINTS - game

- GET /game/creategame
  - Authorization: header Bearer carries JWT
  - Load **mode** from **header** (True for PVP and False for PVE)
  - Create game room w/ related table object
  - Return gameid
- GET /game/joingame
  - Authorization: header Bearer carries JWT
  - Load **gameid** from **header** (True for PVP and False for PVE)
  - Update gameroom related table object(p2)
  - Return message ‘Join Success’

API ENDPOINTS - game continue

- POST /game/redraw [rid, player, redraw_position]
  - Authorization: header Bearer carries JWT
  - Redraw selected card
  - Return message ‘Redraw success’, new hand deck, remaining redraws
- POST /game/confirm [rid, player]
  - Authorization: header Bearer carries JWT
  - Mark user as ready in round table
  - Return message ‘success’
- POST /game/track [rid, player]
  - Authorization: header Bearer carries JWT
  - Request game progress (no longer used, covered by socketio)
  - Return handdeck, redraw remaining, p1_score, p2_score

API ENDPOINTS - other

- POST /model/train
  - No special authentication (debug purpose/admin only)
  - API to do an instant update of Q-table
  - Return game_processed for qtable update.
- WebSocket: listen [gameid, playerid, JWT]
  - Authorization: JWT from JSON
  - Uses ‘system’ event to broadcast game status information based on room separated by gameid.

IN CONCLUSION…

Domain-Specific Detail

- Real-time communication layer using Flask_socketio for synchronized gameplay between players.
- Hybrid architecture combining REST APIs (setup/actions), Flask_socketio (live updates), and DynamoDB (persistent game state).
- Reusable frontend state management enabling both PVP and PVE using the same interface without code duplication.

OUTCOMES AND RESULTS

- Success
  - The frontend is fully functional end-to-end, with all pages rendering correctly and UI states updating in real time.
  - The frontend integrates reliably with the backend through REST APIs and Flask_socketio, ensuring synchronized gameplay.
  - Real-time Gameplay: Stable PvP & PvE modes powered by Flask_socketio.
  - AI integration: Q-Learning agent successfully trained and deployed in game
  - Asynchronous data processing: Process training data daily to update our Q-Learning agent
- Failure
  - Single-threaded backend cannot handle high frequent requests, resulting in efficiency limitations

INCOMPLETE AND DESCOPED FEATURES

- One thing we consider but never implemented is the scoring system
  - Specifically, an systematic score evaluation of hand decks.
- This is not implemented due to two main reasons:
  - Q-table state limitation
  - Difficult to provide a complex and fair scoring system
- Better Approach
  - Choose one existing card game to implement instead of creating one